

UPSC Adaptive AI Orchestrator

Master System Documentation — Prompt-Ready Reference

Version: 1.0 — Architecture Locked

Status: Pre-Implementation. All decisions finalized.

Purpose: Single source of truth for generating all code prompts. Every architectural decision, mathematical model, schema definition, and module boundary is recorded here. No assumptions remain open.

Document Purpose

This document supersedes the original PDF prompt repository. It incorporates all architectural corrections, schema revisions, and resolved ambiguities from the design review pass. Use this as the primary context document when writing or executing any module generation prompt. The original PDF should not be referenced for implementation — this document takes precedence on all conflicting points.

Part I: System Identity

1.1 Objective

Build a **local-first, cloud-assisted adaptive study engine** for a single UPSC Civil Services candidate. The system eliminates passive study by replacing static reading with dynamically generated, trap-laden questions calibrated to actual UPSC examination patterns. It maintains persistent knowledge state across sessions, adapts question difficulty using an Elo-style rating system, grades open-ended Mains answers through an adversarial multi-pass pipeline, and guarantees daily test availability even during cloud API failure.

1.2 Design Philosophy

- **Local-first:** All persistent state lives on local disk. Cloud is a reasoning accelerator, not a dependency for data continuity.
- **Stateless spokes:** Every module except the database layer is stateless. Modules read from and write to the Hub. They do not hold state between calls.
- **Mathematical adaptation:** All scheduling, difficulty targeting, and performance measurement is driven by explicit mathematical formulas — not heuristics.
- **Graceful degradation:** The system must always be able to deliver a 30-question test, even if the cloud API is unavailable.
- **Transparency:** Every generated test must document its own composition logic so the student understands why specific questions were chosen.

1.3 Operational Parameters

Sprint Type	Questions	Time	Frequency
GS Prelims	30	30 minutes	Daily
CSAT	30	30 minutes	Daily
Mains	4 analytical	30 minutes (7.5 min/answer)	Daily
Diagnostic	25 (5 per subject)	One-time onboarding	Cold start only

Backlog Recovery Rule: If a daily sprint is skipped, that day's topics enter the `backlog_queue`. Future tests draw 35% of their dynamic static quota from backlog topics and 65% from today's active topic until the backlog is cleared.

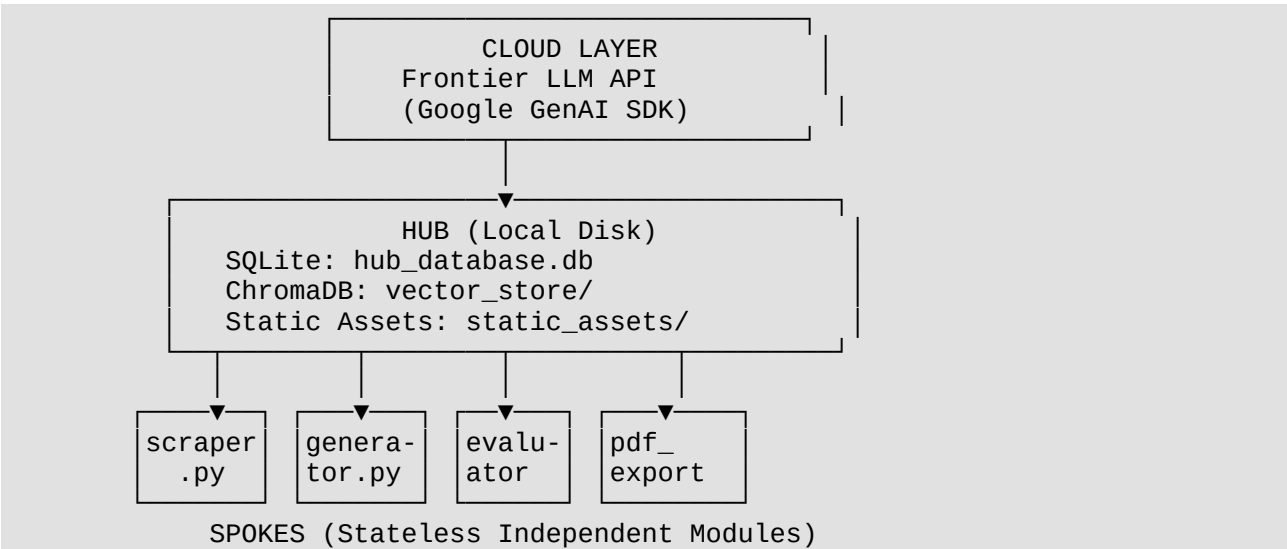
1.4 Hardware Context

- **GPU:** NVIDIA RTX 3050, 6GB VRAM
- **RAM:** 16GB
- **Embedding model selected:** `sentence-transformers/all-MiniLM-L6-v2`
- **Rationale:** Lightweight and fast for local ChromaDB indexing. Upgrade path to `all-mpnet-base-v2` documented for V2 if retrieval quality needs improvement.

Part II: Architecture

2.1 Architectural Style

Hub-and-Spoke with Cloud Reasoning Layer



2.2 Three-Layer Model

Layer	Components	Responsibility
Hub (Local)	SQLite, ChromaDB, static_assets/	All persistent state. Relational data, vector embeddings, static PYQ pools.
Spokes (Local)	scraper, generator, evaluator, pdf_exporter, diagnostic, safe_mode, benchmark_runner	Stateless execution modules. Read from Hub, write to Hub, call Cloud when needed.
Cloud	Google GenAI SDK	Complex reasoning: question synthesis, adversarial grading, AI synthesis of news articles.

2.3 Directory Structure

UPSC_Agent_Data/	
├─ hub_database.db	← SQLite (all relational tables)
├─ vector_store/	
│ └─ syllabus_collection/	← ChromaDB: static UPSC syllabus content
(60% static RAG)	
│ └─ pyq_collection/	← ChromaDB: PYQ embeddings 2013-2025
(Critic Agent)	
├─ static_assets/	
│ └─ pyq/	
│ └─ prelims/	← JSON files, one per year,
GeneratedQuestionSchema format	
│ └─ mains/	← Plain text files, one per year
└─ topper_copies/	← Plain text files of recognized Mains
topper answers	
├─ exports/	
│ └─ Question_Paper.pdf	
│ └─ Answer_Key_and_Analysis.pdf	
│ └─ Daily_Current_Affairs_Briefing.pdf	
├─ archive/	← Cold data (90+ days compressed)
└─ calibration_config.yaml	← All hyperparameters

2.4 Module Registry

Module File	Phase	Prompt #	Status
calibration_config.yaml	0	Prompt 8	Exists in original doc
calibration.py	0	Prompt 8	Exists in original doc
database.py	0	Prompt 1	Exists — schema updated
models.py	0	Prompt 1 + 10 updates	Exists — schema updated
schemas.py	0	Prompt 2 + addendum update	Exists — CriticEvaluationSchema added
rag_store.py	1	New prompt	Does not exist — must

Module File	Phase	Prompt #	Status
			be written
scraper.py	1	Prompt 6 (was missing)	Does not exist — must be written
generator.py	2	Prompt 3 + addendum updates	Exists — composition logic corrected
evaluator.py	2	New prompt	Does not exist — must be written
diagnostic.py	3	Prompt 0	Exists in original doc
pdf_exporter.py	4	Prompt 4 + addendum updates	Exists — three-document version
safe_mode.py	5	Prompt 12	Exists in original doc
main.py	5	Prompt 5 / Update Prompt 7	Exists — /submit-answer added
benchmark_runner.py	6	Prompt 9	Exists in original doc

Part III: Development Phases

Build phases enforce dependency order. A phase must be complete before the next begins.

Phase 0 — Foundation

Objective: Establish all persistent infrastructure that every other module depends on.

No external dependencies. All modules in this phase can be built in any order relative to each other.

Phase 0 Modules

calibration_config.yaml + calibration.py

Centralized hyperparameter store and loader. All mathematical constants live here. Spokes import from calibration.py, never from raw YAML. Loader caches config in memory. Supports live-reload on file modification without server restart.

database.py + models.py

SQLAlchemy engine, session factory, and all ORM table definitions. Includes all tables from the original document plus schema additions from review. Auto-initializes on startup.

schemas.py

All Pydantic v2 validation schemas enforcing structured JSON output from the cloud LLM API. Any data received from the cloud must pass schema validation before touching the database.

Phase 0 Outputs

- `hub_database.db` initialized with all tables
 - `calibration_config.yaml` with all resolved values
 - All Python schemas importable
-

Phase 1 — Data Infrastructure

Objective: Build the two primary data ingestion systems: vector storage and news scraping.

Depends on: Phase 0 complete.

Phase 1 Modules

`rag_store.py`

Initializes two ChromaDB collections. Provides retrieval interface for `generator.py`. Population of collections is a one-time ingestion step triggered manually, separate from runtime operation.

`scraper.py` (Prompt 6)

Two-mode scraper. Daily news mode runs on APScheduler schedule. Document ingestor mode runs on event trigger. Both write to `current_affairs_feed`. Neither stores state — they are pure write operations to the Hub.

Phase 1 Outputs

- `vector_store/syllabus_collection/` populated
 - `vector_store/pyq_collection/` populated
 - `current_affairs_feed` receiving daily records
-

Phase 2 — Core Intelligence

Objective: Build the question generation and answer evaluation engines.

Depends on: Phase 0 + Phase 1 complete.

Phase 2 Modules

`generator.py`

Async question generation. Applies composition pipeline. Calls cloud API via `asyncio.gather` (3 parallel workers × 10 questions). Runs each draft question through the Critic Agent. Updates `question_bank` and `student_profile Elo` on completion.

`evaluator.py`

Dual-pass adversarial Mains answer grader. Takes a `question_id` and student response text. Pass 1 checks blueprint completeness. Pass 2 adopts adversarial persona. Returns structured score out of 10. Writes to `attempt_history`.

Phase 2 Outputs

- Generator capable of producing 30-question exam papers
 - Evaluator capable of scoring subjective Mains answers
-

Phase 3 — Onboarding

Objective: Build the cold-start diagnostic that populates initial Elo ratings.

Depends on: Phase 0 + Phase 1 + Phase 2 complete.

Phase 3 Modules

`diagnostic.py`

One-time execution script. Runs 25-question diagnostic across 5 subjects. Calibrates initial Elo ratings. Tags weaknesses. Populates `student_profile` before the daily sprint loops begin.

Phase 3 Outputs

- `student_profile` populated with calibrated starting Elo per subject
 - `weakness_tags` populated per subject
 - `baseline_elo_rating` set per subject
-

Phase 4 — Export

Objective: Build the document generation engine.

Depends on: Phase 0 only (reads from database, no generation dependency).

Phase 4 Modules

`pdf_exporter.py`

Three-document ReportLab renderer. Watermarked. Multi-page cursor management. Reads from `question_bank` and `current_affairs_feed`. Writes three PDF files to `exports/`.

Phase 4 Outputs

- `Question_Paper.pdf`
 - `Answer_Key_and_Analysis.pdf`
 - `Daily_Current_Affairs_Briefing.pdf`
-

Phase 5 — Orchestration

Objective: Wire all spokes into a single operational FastAPI server.

Depends on: All previous phases complete.

Phase 5 Modules

safe_mode.py

Circuit breaker. Health-checks cloud API and current_affairs_feed before every test generation cycle. Flags SAFE_MODE_ACTIVE if either is degraded. Reroutes generation to local static_assets/pyq/ pool.

main.py

FastAPI application. All API routes. APScheduler registration for scraper and data archival jobs. Startup event initializes database tables. All business logic is delegated to spokes — main.py is routing only.

Phase 5 Outputs

- Running FastAPI server
 - All endpoints live
 - Scraper running on schedule
 - Data archival running on schedule
-

Phase 6 — Observability

Objective: Add drift detection and quality monitoring.

Depends on: All previous phases complete and running.

Phase 6 Modules

benchmark_runner.py

Standalone validation script. Not part of the API server. Run manually or on extended schedule. Detects generation quality drift against PYQ benchmarks. Detects evaluation drift against topper copies. Outputs drift warnings to terminal.

Phase 6 Outputs

- Automated drift detection capability
 - System Drift Warnings on quality degradation
-

Part IV: Complete Database Schema

All tables are defined in `models.py`. Foreign key relationships are enforced. Session management is handled by a context-managed session factory in `database.py`.

4.1 student_profile

Subject-level tracking. One record per UPSC subject.

Column	Type	Default	Notes
subject_id	Text	—	Primary Key
subject_name	Text	—	e.g., "Polity", "Economy"
current_elo_rating	Integer	1200	Updated after every attempt
baseline_elo_rating	Integer	Set at diagnostic	NEW. Reference point for Recovery Velocity. Set at cold-start calibration. Updated only after 20 consecutive attempts without Elo drop > 50 points.
recovery_velocity_score	Float	0.0	$V_{rec} = \Delta \text{Elo} / \Delta t$. Tracks cognitive endurance.
total_questions_attempted	Integer	0	Cumulative counter
weakness_tags	Text	'[]'	Serialized JSON array of subtopic labels
last_reviewed_at	Timestamp	—	Most recent review

Subjects (initial): Polity, Economy, History, Geography, CSAT

4.2 topic_progress — NEW TABLE

Topic-level tracking within subjects. One record per discrete topic. More granular than student_profile.

Column	Type	Default	Notes
topic_id	Text	—	Primary Key
subject_id	Text	—	Foreign Key → student_profile
topic_name	Text	—	e.g., "Fundamental Rights", "Monetary Policy"
base_stability_index	Float	3.0	I_base for Memory Decay Formula. Matches base_revision_interval_days in config.yaml.

Column	Type	Default	Notes
times_reviewed	Integer	0	Cumulative successful recall count
mistake_count	Integer	0	Cumulative incorrect recall count. Higher values compress next_review_due interval.
last_reviewed_at	Timestamp	—	Most recent review timestamp
next_review_due	Timestamp	—	Computed output of Memory Decay Formula

Design note: No Elo rating in this table for V1. Subject-level Elo in student_profile is the sole Elo reference. Topic-level Elo may be added in V2 if subject-level granularity proves insufficient. The upgrade path is clean: add `topic_elo_rating` Integer column to this table with no other structural changes required.

4.3 daily_study_log

Calendar of scheduled topics. One record per calendar day.

Column	Type	Default	Notes
date_string	Text	—	Primary Key, format: YYYY-MM-DD
core_gs_topic	Text	—	Foreign Key → student_profile.subject_id
csat_topic	Text	—	—
status	Text	'PENDING'	Values: PENDING, COMPLETED, SKIPPED
hours_logged	Float	0.0	—
created_at	Timestamp	—	—

Backlog trigger: When status transitions to 'SKIPPED', the topic is written to backlog_queue.

4.4 backlog_queue

Priority queue of topics missed due to skipped days.

Column	Type	Default	Notes
topic_id	Text	—	Primary Key
topic_type	Text	—	'GS' or 'CSAT'
date_skipped	Text	—	YYYY-MM-DD
priority_weight	Integer	1	Increases each day the topic remains uncleared
times_tested_in_backlog	Integer	0	How many times this topic has appeared in a backlog draw

4.5 current_affairs_feed

Scraped and AI-processed news articles.

Column	Type	Default	Notes
article_id	Text	—	Primary Key
source	Text	—	Source URL or publication name
title	Text	—	Article headline
raw_content	Text	—	Scraped full text
syllabus_mapping	Text	—	e.g., "GS-3 Economy - Banking Sector"
ai_synthesis	Text	—	LLM-generated plain-language summary
fetches_at	Timestamp	—	—
test_generated_flag	Boolean	False	True after questions are generated from this article

4.6 question_bank

All generated questions with full metadata.

Column	Type	Default	Notes
question_id	Text	—	Primary Key
subject_id	Text	—	Foreign Key → student_profile
source_type	Text	—	'STATIC_RAG', 'DYNAMIC_CA',

Column	Type	Default	Notes
			'INTEGRATED'
question_type	Text	—	'PRELIMS_GS', 'CSAT', 'MAINS_SUBJECTIVE'
difficulty_level	Integer	—	1–10 scale
question_text	Text	—	Full question stem
metadata_json	Text	—	Serialized full GeneratedQuestionSchema payload
correct_key	Text	NULL	Answer letter for MCQs. Null for Mains.
provenance_tags	Text	—	Serialized JSON. Fields: data_source, source_fetch_date, generation_model_version, critic_agent_consensus_score, active_adaptation_version
generation_time_ms	Integer	—	Telemetry
tokens_consumed	Integer	—	Telemetry
critic_retry_count	Integer	0	How many Critic Agent rejections before acceptance
created_at	Timestamp	—	—

4.7 attempt_history

Every student response with evaluation and telemetry.

Column	Type	Default	Notes
attempt_id	Text	—	Primary Key
question_id	Text	—	Foreign Key → question_bank
session_id	Text	—	NEW. UUID from test generation. Groups attempts within one test sprint for Drift Index

Column	Type	Default	Notes
			computation.
student_response	Text	—	—
confidence_level	Text	—	NEW. 'HIGH', 'MEDIUM', or 'LOW'. Captured at submission.
response_duration_seconds	Float	—	NEW. Elapsed seconds from question display to submission.
score_percentage	Float	—	0.0–1.0 for MCQ. 0–10 for Mains (stored as float).
detailed_evaluation	Text	—	Markdown breakdown. Stripped at 30-day archival.
thinking_pattern_score	Integer	—	—
evaluation_time_ms	Integer	—	Telemetry
tokens_consumed	Integer	—	Telemetry
attempted_at	Timestamp	—	—

4.8 experiment_runs

Config version history for observability.

Column	Type	Default	Notes
experiment_id	Text	—	Primary Key
config_version	Text	—	YAML version label
benchmark_score	Float	—	Score from benchmark_runner at time of config
run_date	Timestamp	—	—
engineering_notes	Text	—	Why this config change was made

4.9 manual_overrides

Human correction loop.

Column	Type	Default	Notes
override_id	Text	—	Primary Key
target_id	Text	—	FK to either attempt_history or question_bank
override_type	Text	—	'EVALUATION_DISP UTE', 'QUESTION_REJECT ED', 'TOPIC_FORCED'
user_correction _notes	Text	—	—
timestamp	Timestamp	—	—

4.10 Schema Relationships

```
student_profile
├── topic_progress (subject_id FK)
├── daily_study_log (core_gs_topic FK)
├── question_bank (subject_id FK)
│   ├── attempt_history (question_id FK)
│   └── manual_overrides (target_id FK)
current_affairs_feed
└── question_bank (source_type = 'DYNAMIC_CA')
```

Part V: Pydantic Schema Architecture

All schemas live in `schemas.py`. All are Pydantic v2. All fields carry explicit `Field()` descriptions to guide the cloud LLM's output structure.

5.1 PrelimsOptionSchema

MCQ answer choices.

Field	Type	Description
A	str	Option A text
B	str	Option B text
C	str	Option C text

Field	Type	Description
D	str	Option D text

5.2 TrapAnalysisSchema

Cognitive trap embedded in incorrect options.

Field	Type	Description
trap_type	str	e.g., 'Absolute Modifier', 'Chronological Inversion', 'Institutional Swapping'
trap_mechanism	str	How an unprepared student will be misled by the phrasing
elimination_clue	str	The exact logical flaw or wording choice that reveals the trap

5.3 ExplanationSchema

Plain-language solution breakdown.

Field	Type	Description
simple_core_concept	str	High-yield summary of underlying concept in plain language, free of academic jargon
step_by_step_justification	List[str]	Why each specific statement or option is correct or incorrect

5.4 GeneratedQuestionSchema

Full question payload. Every question generated by the cloud API must conform to this schema.

Field	Type	Default	Description
question_text	str	—	Multi-statement question stem
question_type	str	—	Restricted to: 'PRELIMS_GS', 'CSAT', 'MAINS_SUBJECTIV

Field	Type	Default	Description
			E'
difficulty_tier	int	—	1–10 calibration scale
options	Optional[PrelimsOptionSchema]	None	None for Mains subjective
correct_key	Optional[str]	None	Answer letter (A/B/C/D). None for Mains.
explanation_data	ExplanationSchema	—	Always required
trap_data	TrapAnalysisSchema	—	Always required. At least one trap must be deliberately designed.

5.5 CriticEvaluationSchema — NEW

Critic Agent's structured scoring output. Every draft question passes through this schema before database write.

Field	Type	Description
semantic_authenticity_score	float	0.0–1.0. Does the question feel like authentic UPSC?
distractor_plausibility_score	float	0.0–1.0. Are incorrect options genuinely believable?
fact_check_verification_score	float	0.0–1.0. Are all stated facts accurate?
blueprint_alignment_score	float	0.0–1.0. Does the question map cleanly to UPSC syllabus structure?
combined_score	float	Simple average of all four dimension scores
rejection_reason	Optional[str]	None if accepted. Populated if any gate fails.

Part VI: Mathematical Models

All parameters are resolved. All constants are stored in `calibration_config.yaml` and read via `calibration.py`.

6.1 Elo Rating System

Updates `student_profile.current_elo_rating` after every MCQ attempt.

Expected score:

$$E = 1 / (1 + 10^{((R_{\text{question}} - R_{\text{old}}) / 400)})$$

Rating update:

$$R_{\text{new}} = R_{\text{old}} + K \times (S - E)$$

Parameters:

- $K = 32$ (from config: `elo_system.k_factor`)
- R_{old} = current subject Elo from `student_profile`
- R_{question} = `question_bank.difficulty_level` mapped to Elo scale (`difficulty_tier × 100 + 1000` recommended mapping)
- S = Certainty-Weighted Performance score P_w (see 6.2), not raw binary accuracy

Floor: Elo cannot drop below `elo_system.floor_rating = 800`.

6.2 Certainty-Weighted Performance Vector

Adjusts raw binary accuracy before feeding into Elo calculation. Prevents guessing bias from inflating ratings.

$$P_w = S \times (C_f \times (1 - \Delta T / T_{\text{max}}))$$

Parameters:

- S = binary accuracy (1 for correct, 0 for incorrect)
- C_f = confidence coefficient from `certainty_weights` in config:
 - HIGH = 1.0
 - MEDIUM = 0.75
 - LOW = 0.5
- ΔT = deviation from optimal pacing: `|response_duration_seconds - expected_seconds|`
- T_{max} = `certainty_weights.pacing_max_seconds = 120`

Behavior: A correct answer with LOW confidence and significant time overrun scores significantly less than a correct answer with HIGH confidence within pacing bounds. This prevents the system from falsely upgrading a student who guesses correctly.

6.3 Memory Decay Formula

Calculates `topic_progress.next_review_due`. Pulls high-difficulty topics back into rotation sooner.

$$I_{next} = I_{base} \times e^{(\alpha \times C_f \times (2 - D_t))}$$

Parameters:

- `I_next` = next revision interval in days
- `I_base` = `topic_progress.base_stability_index` (default 3.0, updated after each review)
- `α` = `memory_decay.alpha_multiplier` = 1.0 (from config)
- `C_f` = confidence coefficient at most recent attempt (from `attempt_history`)
- `D_t` = `topic_progress.mistake_count` modifier derived from difficulty tier (1–10 scale)

Behavior: Higher difficulty topics and topics with higher `mistake_count` decay faster (shorter `I_next`), pulling them into active rotation sooner. Topics with high confidence recall decay slower (longer `I_next`), reducing unnecessary repetition.

6.4 Recovery Velocity Index

Measures how fast a student recovers their Elo after a subject failure. Tracks cognitive endurance, not just raw knowledge.

$$V_{rec} = \Delta Elo / \Delta t$$

Parameters:

- `V_rec` stored in `student_profile.recovery_velocity_score`
- `ΔElo` = Elo recovered from trough back to `baseline_elo_rating`
- `Δt` = days elapsed from trough to recovery

Trigger: Fires when `current_elo_rating` drops below `baseline_elo_rating` for more than 3 consecutive attempts in a subject.

Baseline update rule: `baseline_elo_rating` is updated only after 20 consecutive attempts without a downward Elo correction exceeding 50 points. It is not updated on single-session fluctuations.

6.5 Cognitive Pacing Index

Tracks within-session response velocity to detect fatigue and panic guessing.

Computation: Query `attempt_history` WHERE `session_id` = ?. Calculate standard

deviation of `response_duration_seconds` across the session's attempts.

Trigger: If pacing standard deviation exceeds `behavioral_fatigue_limits.pacing_std_dev_threshold = 0.40` within a single session, the Psychological Drift Index is flagged.

Response: Instead of increasing difficulty, the orchestrator intercepts the schedule and injects an anchored review sprint or recommends a mandatory break.

Topic avoidance detection: If a `daily_study_log` entry remains PENDING past a threshold window while the student has been active in other topics, it is flagged as an avoidance event and logged.

Part VII: Module Specifications

For each module: responsibility, key inputs, key outputs, internal dependencies, and critical implementation notes.

7.0 `calibration.py` + `calibration_config.yaml`

Responsibility: Centralize all algorithmic hyperparameters. Provide a cached, importable Python interface.

Key behavior:

- Parses YAML on first import, caches result in memory
- Supports live-reload: if YAML file is modified, caller can request fresh parse without server restart
- All spokes import constants from this module, never from raw YAML or hardcoded values

Complete `calibration_config.yaml` structure:

```
elo_system:
  k_factor: 32
  base_rating: 1200
  floor_rating: 800

certainty_weights:
  high: 1.0
  medium: 0.75
  low: 0.5
  pacing_max_seconds: 120

current_affairs_filters:
  similarity_threshold: 0.88
  rolling_window_hours: 48
  min_source_consensus: 2

memory_decay:
  alpha_multiplier: 1.0
  base_revision_interval_days: 3.0
  difficulty_weight_scaler: 0.15
```

```
curricular_floor:
    random_syllabus_allocation: 0.20

behavioral_fatigue_limits:
    pacing_std_dev_threshold: 0.40
    streak_decay_trigger: 3

critic_thresholds:
    fact_check_verification: 0.85
    semantic_authenticity: 0.75
    distractor_plausibility: 0.75
    blueprint_alignment: 0.70
    combined_minimum: 0.85

diagnostic:
    questions_per_subject: 5
    exceptional_threshold: 0.85
    exceptional_elo: 1450
    average_elo: 1200
    growth_elo: 950
    average_threshold: 0.50

scraper:
    daily_run_hour: 6
    daily_run_minute: 0
    archival_hot_days: 30
    archival_warm_days: 90

pdf:
    watermark_text: "THE FELLOW ASPIRANT"
    watermark_font_size: 52
    watermark_opacity: 0.15
    watermark_rotation: 45
    bottom_buffer: 120
    mains_blank_space: 90
```

7.1 database.py + models.py

Responsibility: SQLAlchemy engine, session factory, and all ORM table definitions.

Key behaviors:

- Engine connects to UPSC_Agent_Data/hub_database.db
- Creates directory if it does not exist
- `init_db()` function creates all tables on first run (idempotent)
- Context-managed session generator for all spoke usage

Tables defined: student_profile, topic_progress, daily_study_log, backlog_queue, current_affairs_feed, question_bank, attempt_history, experiment_runs, manual_overrides

Custom Exception Classes (also defined here or in a shared exceptions module):

- GenerationFailure
- EvaluationFailure

- CalibrationFailure
 - CurrentAffairsFailure
 - RAGFailure
 - UserBehaviorFlag
-

7.2 schemas.py

Responsibility: Pydantic v2 validation of all cloud LLM API responses.

Schemas defined: PrelimsOptionSchema, TrapAnalysisSchema, ExplanationSchema, GeneratedQuestionSchema, CriticEvaluationSchema

Critical note: Every field in every schema must carry an explicit `Field(description="...")` metadata string. This description is read by the cloud LLM to understand what it must produce. Vague descriptions produce vague outputs.

7.3 rag_store.py — NEW MODULE

Responsibility: Initialize and interface with ChromaDB. Provide text chunk retrieval for generator.py and PYQ similarity retrieval for the Critic Agent.

Two collections:

Collection	Content	Consumer
syllabus_collection	UPSC static syllabus text chunks	generator.py (60% static RAG content)
pyq_collection	PYQ question embeddings, 2013–2025	Critic Agent in generator.py

Embedding model: sentence-transformers/all-MiniLM-L6-v2 (local, no API call)

Key functions:

- `initialize_collections()` — Creates collections if they do not exist
- `ingest_syllabus_documents(directory_path)` — One-time ingestion from local text files
- `ingest_pyq_documents(directory_path)` — One-time ingestion from `static_assets/pyq/`
- `retrieve_syllabus_chunks(topic: str, n_results: int)` → List of text strings
- `retrieve_similar_pyqs(question_text: str, n_results: int)` → List of PYQ records

Population: Both `ingest_*` functions are called manually once before the system is first used. They are not called at runtime. Runtime only calls `retrieve_*` functions.

7.4 scraper.py — PROMPT 6 (was missing)

Responsibility: Populate `current_affairs_feed` with processed, syllabus-mapped news articles.

Two-mode architecture. These are separate functions. They do not share execution logic.

Mode 1: Daily News Scraper

Schedule: 06:00 daily (registered with APScheduler in `main.py`)

Sources by tier:

Tier	Sources	Consensus Requirement
Tier 1	PIB (Press Information Bureau), PRS Legislative Research, Government portals	1 source sufficient (primary official sources)
Tier 2	The Hindu, Indian Express	2-source consensus within 48-hour rolling window

Consensus filter logic (for Tier 2):

1. Fetch article → write to staging cache (not yet to `current_affairs_feed`)
2. Check if a matching policy issue appears in Tier 1 or another Tier 2 source within 48 hours
3. If yes → approved for processing
4. If no → hold in staging, check again at next scrape cycle

Deduplication logic (all tiers):

1. Compute cosine similarity between incoming article and existing approved articles in same 48-hour window
2. If $\text{similarity} \geq \text{current_affairs_filters.similarity_threshold}$ (0.88) → merge content into single consolidated record, do not create duplicate

Post-approval processing:

1. Send approved article to cloud LLM API → generate `ai_synthesis` (plain-language summary) and `syllabus_mapping`
2. Write complete record to `current_affairs_feed` with `test_generated_flag = False`

Mode 2: Document Ingestor

Schedule: Event-triggered on publication (RBI releases, Economic Survey, Budget documents)

Sources: RBI releases, Economic Survey, Annual Budget documents

Behavior: Downloads full document, splits into sections, processes each section through the same AI synthesis step, writes to `current_affairs_feed`

No consensus filter required for Tier 3 — these are primary government publications.

7.5 generator.py

Responsibility: Compose and generate 30-question exam papers. Run Critic Agent quality gate. Update Elo ratings.

Key functions:

`build_composition_plan(subject_id, test_type)` → Returns composition dictionary

`generate_full_adaptive_exam(composition_plan, context_text)` → Returns `List[GeneratedQuestionSchema]`

`fetch_question_segment(batch_size, context, system_instruction)` → Calls cloud API, returns 10 questions

`run_critic_agent(draft_question, pyq_similar)` → Returns `CriticEvaluationSchema`

`calculate_elo_update(subject_id, P_w, difficulty_tier)` → Updates `student_profile`

Composition pipeline (mandatory sequential order — document this in code comments):

```
Step 1: Apply floor guarantee first
        floor_count = round(30 × curricular_floor.random_syllabus_allocation) → 6 questions
        Draw floor_count topics from random syllabus distribution
        Overlap with today's topic is ALLOWED — do not redraw
        Content type (static or CA) follows topic's natural category

Step 2: Recalculate remaining quota dynamically
        remaining = 30 - floor_count → 24 questions

Step 3: Apply 60/40 content type split to remaining quota
        static_count = round(remaining × 0.60) → ~14 questions
        ca_count = remaining - static_count → ~10 questions

Step 4: Within each category, apply backlog rule if backlog_queue is non-empty
        static_backlog = round(static_count × 0.35)
        static_today = static_count - static_backlog
        ca_backlog = round(ca_count × 0.35)
        ca_today = ca_count - ca_backlog
        If no backlog: static_today = static_count, ca_today = ca_count
```

Parallel batch execution:

`asyncio.gather(fetch_question_segment × 3)` — Three concurrent workers, each requesting 10 questions. Prevents output token ceiling hits.

Critic Agent pipeline (per question):

1. Generate draft question

2. Retrieve top-3 similar PYQs from `pyq_collection` via `rag_store.py`
3. Call cloud API with draft + PYQs → returns `CriticEvaluationSchema`
4. Check Gate 1: All per-dimension floors from `critic_thresholds` in config
5. Check Gate 2: `combined_score ≥ critic_thresholds.combined_minimum`
6. If either gate fails → regenerate (max 3 retries)
7. If 3 retries exhausted → pull from `static_assets/pyq/` anchored pool
8. Write `critic_agent_consensus_score` to `provenance_tags`

System instruction to cloud LLM for question generation must include:

- Behave as a senior UPSC question setter
- Craft multi-layered, non-linear questions designed to penalize rote memorization
- Deliberately design at least one classic civil services cognitive trap within incorrect options
- Populate `trap_data` and `explanation_data` fully
- `simple_core_concept` must be accessible, direct, and jargon-free
- `trap_mechanism` must explicitly state how an unprepared student will be misled

Syllabus Floor Guarantee (hardcoded): 20% of questions in every sprint are drawn from random syllabus distribution regardless of backlog size or weakness tags. This is implemented as Step 1 of the composition pipeline and is not overridable by any adaptive rule.

System Intent Header: Every generated test dictionary must compile a structured header documenting composition parameters. Example: "Today's Test Architecture: 6 Floor (Random Syllabus), 14 Static Core [Polity: Pacing Correction], 10 Current Affairs [Economy: Backlog Recovery]". Written to the test record and displayed during review.

7.6 evaluator.py — NEW MODULE

Responsibility: Grade Mains subjective responses using a dual-pass adversarial pipeline.

Input: `question_id`, `student_response` (text), `confidence_level`, `response_duration_seconds`

Output: Structured evaluation written to `attempt_history`. Score out of 10.

Pass 1 — Anchor Validation:

- Retrieve the question's `blueprint` from `question_bank.metadata_json`
- Prompt cloud LLM to check response against mandatory elements: constitutional articles, committee names, institutional data points, structural markers
- LLM returns a structured pass/fail per blueprint element with a partial score

Pass 2 — Adversarial Cross-Examination:

- Feed Pass 1 results + original response back to cloud LLM
- LLM adopts a critical adversarial examiner persona
- Scans for: unaddressed counter-evidence, logical leaps, circular reasoning, weak policy execution plans, absolute claims without qualification
- Final score is heavily weighted on surviving this cross-examination
- Returns specific argument-level feedback

Final score: Weighted composite of Pass 1 structural completeness and Pass 2 adversarial survival. Score range: 0–10. Stored in `attempt_history.score_percentage`.

Grading bias prevention: The system instruction must explicitly prohibit rewarding verbose prose, Markdown formatting, or generic buzzwords. Evaluation must penalize these if they substitute for substantive argument.

7.7 diagnostic.py

Responsibility: Cold-start profiling. One-time execution only.

Subjects: Polity, Economy, History, Geography, CSAT (5 subjects)

Questions: 5 per subject = 25 total

Question mix per subject: Factual recall + integrated current affairs application + logical interpretation

Epoch tracking: Record exact timestamp when question is displayed and when student submits. Calculate seconds-per-question for Cognitive Pacing Index baseline.

Evaluation:

- MCQ: Calculate strict accuracy percentage per subject
- Subjective: Send to cloud LLM, returns structured JSON with conceptual blind spots, missing institutional keywords, logical fallacy flags

Elo calibration thresholds (from config):

Performance	Elo Assigned
> 85% accuracy with optimal pacing	1450
50–85% accuracy	1200
< 50% accuracy OR severe time overruns	950

Weakness tagging: Parse LLM evaluation output. Convert discovered flaws into standardized subtopic labels. Write serialized JSON array to `student_profile.weakness_tags`. Examples: "Weak Core Memory", "Over-reliance on Absolute Traps", "Missing Structural Keywords", "Pacing Deficit".

Baseline setting: `baseline_elo_rating` set equal to calibration Elo at this point. This is the only time it is set without the 20-consecutive-attempt rule.

Terminal output: Print each subject's baseline Elo and weakness tags to stdout as they are written to disk.

7.8 pdf_exporter.py

Responsibility: Generate three watermarked A4 PDF documents from database records.

Library: ReportLab

Output directory: `UPSC_Agent_Data/exports/` — auto-create if not exists

Watermark (applied to every page of every document):

- Text: "THE FELLOW ASPIRANT"
- Font size: 52
- Color: light grey
- Rotation: 45 degrees diagonal across page center
- Opacity: `setFillAlpha(0.15)` exactly
- Applied to canvas before text is drawn
- Re-applied on every new page after `showPage()`

Multi-page cursor management:

- Track vertical cursor `cursor_y` as text is drawn
- If `cursor_y < 120` (bottom buffer threshold): call `showPage()`, reset cursor to top margin, re-apply watermark

Document 1: Question_Paper.pdf

- Bold question numbers: Q1., Q2., etc.
- Uniformly indent MCQ options: (a), (b), (c), (d)
- For MAINS_SUBJECTIVE questions: leave 80–100 point blank vertical space below for handwritten notes
- No answers, no explanations, no trap analysis in this document

Document 2: Answer_Key_and_Analysis.pdf

For each question:

- Bold header: Q[N] Answer Key: [B]
- Section titled "**Core Concept Explained**": plain-language `explanation_data.simple_core_concept +`

step_by_step_justification

- Distinct block titled "**Trap Analysis & Elimination Strategy**":
trap_data.trap_type, trap_data.trap_mechanism,
trap_data.elimination_clue
- For MAINS_SUBJECTIVE: bulleted list of model evaluation framework points from blueprint

Document 3: Daily_Current_Affairs_Briefing.pdf

- Fetches all records from current_affairs_feed where fetched_at date = today
 - For each record:
 - Bold header: article title
 - Sub-header: syllabus_mapping (e.g., "Syllabus Mapping: GS-3 Economy - Banking Sector")
 - Body: ai_synthesis formatted as clean plain-language briefing
 - Designed for offline reading — no internet access needed after PDF generation
-

7.9 safe_mode.py

Responsibility: Circuit breaker. Protects daily sprint delivery under cloud or scraper failure.

Health check (runs before every test generation):

1. Ping cloud LLM API endpoint with minimal test call
2. Check if current_affairs_feed has records with fetched_at within last 24 hours

Safe mode triggers (any of the following):

- Cloud API timeout or 5xx response
- Cloud API ping fails
- No current affairs records in last 24 hours

When SAFE_MODE_ACTIVE = True:

- Bypass all async cloud API generator calls
- Route test generation to static_assets/pyq/ flat JSON pool
- Serve pre-verified PYQs directly — no ChromaDB required (intentional: ChromaDB itself may be the failure point)
- Return distinct warning flag to API response: "Safe Mode Engaged: API limits reached. Serving offline static database."
- Log safe mode activation to experiment_runs with engineering_notes = "Safe mode triggered: [reason]"

Return to normal: On next test generation request, re-run health check. If health check passes, `SAFE_MODE_ACTIVE` returns to False automatically.

7.10 `main.py`

Responsibility: FastAPI application. Central orchestration hub. All business logic delegated to spokes.

Startup event:

- Call `init_db()` from `database.py` — initializes all tables
- Register APScheduler jobs:
 - `scraper.daily_news_scraper()` → daily at 06:00
 - `archival_hot_to_warm()` → nightly at 02:00
 - `archival_warm_to_cold()` → weekly Sunday at 03:00

API Routes:

Route	Method	Spoke Called
<code>/generate-test/prelims</code>	POST	<code>safe_mode.py</code> → <code>generator.py</code> → <code>rag_store.py</code>
<code>/submit-answer</code>	POST	<code>evaluator.py</code> (MCQ) or direct Elo update
<code>/generate-test/export</code>	POST	<code>pdf_exporter.py</code> (Doc 1 + Doc 2)
<code>/generate-briefing/export</code>	POST	<code>pdf_exporter.py</code> (Doc 3)
<code>/evaluate/mains</code>	POST	<code>evaluator.py</code>

Error handling: All custom exception classes from `models.py` must be caught and logged with specific error codes. Do not return generic 500 responses. Return structured error payloads.

Session ID generation: POST `/generate-test/prelims` generates a UUID `session_id` and returns it to the client. All subsequent `/submit-answer` calls for that session must include this ID.

7.11 `benchmark_runner.py`

Responsibility: Standalone offline drift detection. Not part of the API server. Run manually or on extended schedule.

Part 1 — Generation Validation (PYQ Alignment):

1. Load a held-out subset of PYQs from `static_assets/pyq/` (not used for training)

2. Prompt cloud LLM to analyze authentic PYQs and generate new variations using the standard system prompt
3. Compare semantic density and distractor complexity of generated output against authentic PYQ
4. Log alignment score

Part 2 — Evaluation Validation (Topper Copy Alignment):

1. Load topper Mains essay responses from `static_assets/topper_copies/`
2. Pass through `evaluator.py` dual-pass pipeline
3. Record scores

Drift Reporting:

- If generated questions score significantly below PYQ semantic density → output "System Drift Warning: Generation quality degraded"
 - If topper copies score below 8.5/10 → output "System Drift Warning: Evaluation calibration degraded — prompt instructions or weights need recalibration"
 - All results written to `experiment_runs` table
-

Part VIII: API Route Specifications

8.1 POST /generate-test/prelims

Request body:

```
{
  "subject_id": string,
  "test_type": "PRELIMS_GS" | "CSAT"
}
```

Orchestration:

1. Run `safe_mode.health_check()` → if fails, route to safe mode
2. Query `daily_study_log` for today's topic and status
3. Query `backlog_queue` for active backlog items
4. Query `rag_store.retrieve_syllabus_chunks(subject_id)` for static context
5. Query `current_affairs_feed` for today's approved articles
6. Call `generator.build_composition_plan()` → apply floor-first pipeline
7. Call `generator.generate_full_adaptive_exam()` → 30 questions

8. Write all questions to `question_bank`
 9. Generate UUID `session_id`
 10. Return question list + `session_id` + System Intent Header
-

8.2 POST `/submit-answer` — NEW ENDPOINT

Request body:

```
{
  "question_id": string,
  "session_id": string,
  "student_response": string,
  "confidence_level": "HIGH" | "MEDIUM" | "LOW",
  "response_duration_seconds": float
}
```

Orchestration:

1. Retrieve question from `question_bank`
 2. If MCQ: compute binary accuracy `S`
 3. Compute `P_w` using Certainty-Weighted formula
 4. Update Elo via `generator.calculate_elo_update()`
 5. Check Recovery Velocity trigger (if Elo drops below baseline for 3+ consecutive attempts)
 6. Check Psychological Drift Index (compute pacing std dev across session)
 7. Write to `attempt_history` (includes `session_id`, `confidence_level`, `response_duration_seconds`)
 8. Update `topic_progress.times_reviewed`, `mistake_count`, `next_review_due`
-

8.3 POST `/generate-test/export`

Request body:

```
{
  "session_id": string
}
```

Orchestration:

1. Retrieve all questions for session from `question_bank`
2. Call `pdf_exporter.generate_question_paper()` → writes `Question_Paper.pdf`
3. Call `pdf_exporter.generate_answer_key()` → writes `Answer_Key_and_Analysis.pdf`

4. Return absolute file paths for both documents
-

8.4 POST /generate-briefing/export

Request body: None (uses today's date automatically)

Orchestration:

1. Query `current_affairs_feed` WHERE `DATE(fetched_at) = today`
 2. Call `pdf_exporter.generate_briefing()` → writes `Daily_Current_Affairs_Briefing.pdf`
 3. Return absolute file path
-

8.5 POST /evaluate/mains

Request body:

```
{
  "question_id": string,
  "session_id": string,
  "student_response": string,
  "confidence_level": "HIGH" | "MEDIUM" | "LOW",
  "response_duration_seconds": float
}
```

Orchestration:

1. Route to `evaluator.py` dual-pass pipeline
 2. Compute Pass 1 anchor score
 3. Compute Pass 2 adversarial score
 4. Compute weighted composite score (0–10)
 5. Write evaluation to `attempt_history`
 6. Return score + detailed feedback markdown
-

8.6 Startup Event

```
@app.on_event("startup")
→ init_db()
→ scheduler.add_job(daily_news_scraper, cron, hour=6)
→ scheduler.add_job(archival_hot_to_warm, cron, hour=2)
→ scheduler.add_job(archival_warm_to_cold, cron, day_of_week='sun', hour=3)
→ scheduler.start()
```

Part IX: Composition Pipeline

This section is the canonical reference for the composition logic in `generator.py`. Future implementations must follow this exact order. Deviating from the sequence creates allocation bugs that are difficult to debug.

9.1 Sequential Algorithm (Annotate in Code)

```
INPUT: subject_id, test_type, total_questions = 30

# -----
# STEP 1: FLOOR GUARANTEE (execute first, always)
# Purpose: Prevent hyper-personalization loops
# -----
floor_rate = config.curricular_floor.random_syllabus_allocation # 0.20
floor_count = round(total_questions * floor_rate)                # 6

floor_topics = random_draw_from_full_syllabus(floor_count)
# RULE: Overlap with today's topic is ALLOWED. Do not redraw.
# RULE: Content type (CA or static) follows topic's natural category.

# -----
# STEP 2: RECALCULATE REMAINING QUOTA DYNAMICALLY
# -----
remaining = total_questions - floor_count # 24

# -----
# STEP 3: CONTENT TYPE SPLIT on remaining quota
# -----
static_count = round(remaining * 0.60) # ~14
ca_count     = remaining - static_count # ~10

# -----
# STEP 4: BACKLOG RULE within each category
# -----
has_backlog = query_backlog_queue(subject_id).count > 0

if has_backlog:
    static_backlog = round(static_count * 0.35) # ~5
    static_today   = static_count - static_backlog # ~9
    ca_backlog     = round(ca_count * 0.35)      # ~3-4
    ca_today       = ca_count - ca_backlog       # ~6-7
else:
    static_today   = static_count
    static_backlog = 0
    ca_today       = ca_count
    ca_backlog     = 0

# -----
# VERIFY: total allocation must equal 30
# -----
assert floor_count + static_today + static_backlog + ca_today + ca_backlog == 30
```

9.2 Example Calculation (with backlog)

floor_count:	6	(random syllabus, overlap allowed)
static_today:	9	(today's GS topic, from ChromaDB RAG)
static_backlog:	5	(backlog topics, from ChromaDB RAG)
ca_today:	6	(today's current affairs articles)
ca_backlog:	4	(backlog current affairs articles)

Part X: Quality Guardrails

All ten guardrails from the original architecture document. Implementation responsibilities assigned.

#	Guardrail	Implemented In
1	Multi-Agent Critique Pipeline	generator.py (Critic Agent loop)
2	Trap Taxonomy Matrix	schemas.py (TrapAnalysisSchema) + generator system prompt
3	Certainty-Weighted Elo	main.py (/submit-answer) + generator.py (Elo update)
4	Consensus-Driven Synthesis Filter	scraper.py (staging cache + deduplication)
5	Dual-Pass Adversarial Mains Evaluation	evaluator.py
6	Psychological Drift Index	main.py (/submit-answer → pacing std dev check)
7	UPSC-Calibrated Spaced Repetition	generator.py (reads topic_progress.next_review_due)
8	Over-Personalization Floor Guarantee	generator.py (Step 1 of composition pipeline)
9	Tiered Data Archival Policy	main.py (APScheduler jobs)
10	System Intent Explainability Layer	generator.py (System Intent Header per test)

Critic Agent Dual-Gate Thresholds (Guardrail #1)

Gate 1 — Per-dimension minimums (all must pass):

Dimension	Minimum Score	Rationale
fact_check_verification	0.85	Factual errors are disqualifying. Hardest floor.
semantic_authenticity	0.75	Question must feel like actual UPSC
distractor_plausibili	0.75	Weak distractors trivialize the

Dimension	Minimum Score	Rationale
ty		difficulty
blueprint_alignment	0.70	Structural imperfection is tolerable. Softest floor.

Gate 2 — Combined average ≥ 0.85

Rejection policy: Either gate fail → regenerate. Max 3 retries → fallback to static PYQ pool.

Part XI: Scraper Architecture Detail

Source Tiers and Consensus Requirements

Tier	Sources	Consensus	Notes
1	PIB, PRS Legislative Research, Government portals	1 source sufficient	Official primary sources. High authority.
2	The Hindu, Indian Express	2 sources within 48 hours	Mainstream national coverage.
3	RBI releases, Economic Survey, Budget documents	No consensus required	Periodic publications, event-triggered.

Two-Mode Execution (must remain separate functions)

Mode 1 — Daily News Scraper (daily_news_scraper):

- Scheduled via APScheduler at 06:00 daily
- Targets Tier 1 and Tier 2 sources
- Applies staging cache → consensus filter → deduplication → AI synthesis → write to current_affairs_feed

Mode 2 — Document Ingestor (document_ingestor):

- Triggered by event (publication release), not schedule
- Targets Tier 3 sources
- Downloads full document → section split → AI synthesis per section → write to current_affairs_feed
- No consensus filter (these are primary official publications)

Both modes share: AI synthesis call, syllabus_mapping generation, write to current_affairs_feed.

Both modes do not share: Scheduling logic, source targets, deduplication logic, section splitting logic.

Part XII: Data Lifecycle Policy

Executed by APScheduler jobs registered in `main.py`.

Phase	Age	Action	Job Schedule
Hot	Days 1–30	Full records, verbose JSON, pacing data — no action	—
Hot → Warm	Day 31	Strip <code>detailed_evaluation</code> from <code>attempt_history</code> . Keep scores, tags, accuracy rates.	Nightly 02:00
Warm → Cold	Day 91	Compress historical rows to flat JSON files in <code>UPSC_Agent_Data/archive/</code> . Delete from SQLite.	Weekly Sunday 03:00

Part XIII: Module Dependency Graph

Read this before writing any prompt. A module must not be written before its dependencies are written.

```
Phase 0 (no dependencies)
├─ calibration_config.yaml
├─ calibration.py           (reads: calibration_config.yaml)
├─ models.py               (reads: nothing)
├─ database.py             (reads: models.py)
└─ schemas.py              (reads: nothing)

Phase 1 (depends on Phase 0)
├─ rag_store.py            (reads: database.py, calibration.py)
└─ scraper.py              (reads: database.py, calibration.py, rag_store.py)

Phase 2 (depends on Phase 0 + Phase 1)
├─ generator.py           (reads: database.py, schemas.py, calibration.py,
rag_store.py)
└─ evaluator.py           (reads: database.py, schemas.py, calibration.py)

Phase 3 (depends on Phase 0 + Phase 1 + Phase 2)
└─ diagnostic.py          (reads: database.py, schemas.py, calibration.py,
generator.py)
```

```
Phase 4 (depends on Phase 0 only)
└─ pdf_exporter.py      (reads: database.py, calibration.py)

Phase 5 (depends on all previous)
└─ safe_mode.py         (reads: database.py, calibration.py, generator.py)
└─ main.py              (reads: ALL modules above)

Phase 6 (depends on all)
└─ benchmark_runner.py  (reads: database.py, evaluator.py, generator.py,
calibration.py)
```

Part XIV: Implementation Notes for Prompt Writing

When writing generation prompts for each module, always include the following context:

1. **State the phase.** Explicitly say which phase this module belongs to and what modules it may import.
 2. **Reference the exact schema.** When a module writes to a database table, paste the column list from Part IV. Do not leave the LLM to infer column names.
 3. **Reference the exact formula.** When a module computes math, paste the formula from Part VI with all parameter names explicitly mapped to their database column or config key source.
 4. **State what is NOT this module's responsibility.** Generator does not grade answers. Evaluator does not compose tests. Scraper does not generate questions. State the boundary explicitly.
 5. **State the composition pipeline order verbatim for generator.py.** Paste the Step 1 through Step 4 comment block from Part IX into the prompt. This is too easy to get wrong.
 6. **For safe_mode.py:** State clearly that the fallback pool reads from flat JSON files in `static_assets/pyq/` and does NOT require ChromaDB or the cloud API. This is intentional.
 7. **For all PDF generation:** Watermark parameters must be stated exactly: font size 52, opacity 0.15, rotation 45, text "THE FELLOW ASPIRANT". These cannot be approximate.
 8. **For schemas.py:** State that every `Field()` must include a `description=` argument. This is not optional — the description is read by the cloud LLM.
-

End of Master System Documentation

Architecture locked. All decisions recorded. Ready for sequential prompt generation.